

INDEX

SI NO	NAME OF EXPERIMENT	DATE	PAGE NO	REMARKS
1	Practice Computer Hardware – Familiarization CPU Box, Motherboard, CPU & Chip-set, Interface cards, Card slots, Hard disk, Cables, SMPS, NIC, Various ports, etc. Computer Peripherals - I/O Devices. Storage devices, Interface cards – Buses – Firmware	28-01-2025	2	
2	Familiarization of Boot process	04-02-2025	8	
3	Familiarizing installation of Linux and Windows operating systems	11-02-2025	14	
4	Familiarizing basic Unix/Linux commands - ls, mkdir, cp, mv, grep, rmdir, chmod, useradd, passwd, history, dmesg, cpuinfo, uname, du, time, write, fdisk	18-02-2025	22	
5	Shell programming in Linux(bash)	25-02-2025	34	
6	Create a web page and deploy on a local web server.	04-03-2025	40	
7	Use Javascript to validate forms.	18-03-2025	46	
8	Create an image slider using HTML, CSS, and JavaScript. Allow users to navigate between images using previous and next buttons.	25-03-2025	50	
9	Familiarisation of LaTeX - Basic only	01-04-2025	56	
10	Familiarisation of Development Environments - Visual studio code, Sublime Text, Atom	08-04-2025	60	
11	Familiarizing Basic Networking Commands	15-05-2025	66	

Experiment 1: Practice Computer Hardware

Objective:

To understand and identify the various internal and external hardware components of a computer system including the CPU box, motherboard, chipset, interface cards, storage devices, power supply units, I/O devices, ports, and cables.

Theory:

A computer system consists of both hardware and software. The hardware components are the physical parts of the computer that you can touch and see. In this experiment, we explore the internal and external hardware components that are essential for the functioning of a computer system.

1. CPU Box (Cabinet/Chassis):

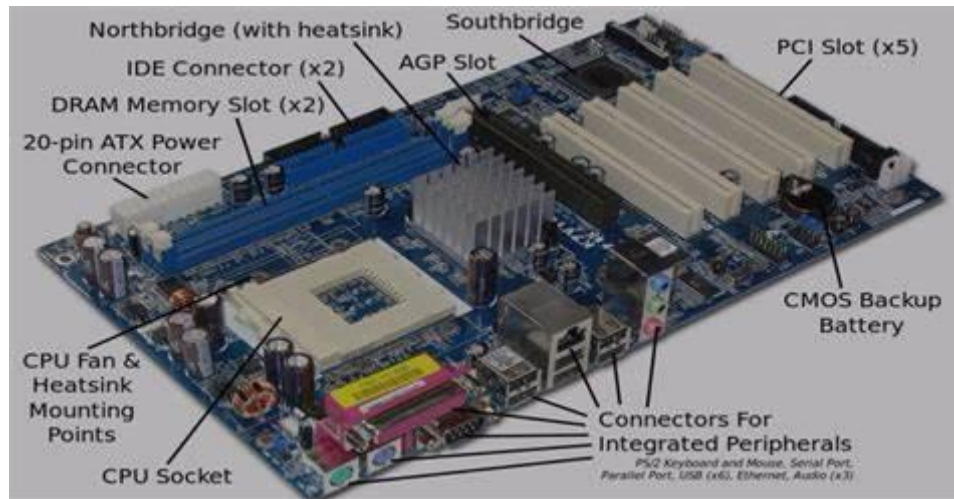
The CPU box is the enclosure that contains the major internal components of the computer. It provides protection and supports airflow and heat dissipation. It houses the motherboard, power supply, storage devices, and other essential parts.

2. Motherboard:

The motherboard is the main circuit board of the computer. It serves as a platform to connect all other components including the CPU, RAM, storage, and peripheral devices.

Key parts on the motherboard include:

- CPU Socket – Holds the processor
- RAM Slots – For installing memory modules
- Chipsets – Manage data flow between components
- Expansion Slots (PCI, PCIe) – For interface cards
- SATA Ports – Connect storage devices
- Power Connectors – From SMPS
- I/O Ports Panel – External connectivity (USB, audio, etc.)



Motherboard



CPU (Central Processing Unit)



SMPS (Switched-Mode Power Supply):

3. CPU (Central Processing Unit):

The CPU is the brain of the computer. It performs arithmetic and logic operations and controls all other components. Modern CPUs may have multiple cores (dual-core, quad-core, etc.) for better performance.

4. Chipset:

The chipset is a group of microchips on the motherboard that manage communication between the processor, memory, and peripheral devices.

- **Northbridge:** Connects CPU with RAM and graphics
- **Southbridge:** Manages input/output devices

5. SMPS (Switched-Mode Power Supply):

SMPS converts AC power from the wall socket into DC power required by the computer's internal components. It distributes regulated power to the motherboard, hard disk, and other devices.

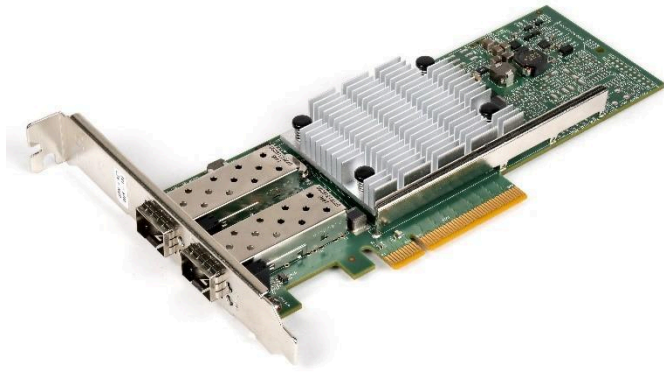
6. Storage Devices:

- **Hard Disk Drive (HDD):** Magnetic storage for data
- **Solid-State Drive (SSD):** Faster flash-based storage
- **Optical Drives:** For CDs/DVDs (now rarely used)
- **External Drives:** Portable USB drives

7. Interface Cards:

Installed into expansion slots, these provide extra functionality:

- **NIC (Network Interface Card):** Network connectivity
- **Graphics Card (GPU):** Video output and rendering
- **Sound Card:** Enhances audio features



Network Interface Card



Hard disc



DVD



CD-ROM



Pendrive



Floppy Disk



Memory Card

8. Ports and Connectors:

Used for connecting peripherals and data transmission:

- **USB Ports** – For devices like pen drives, printers
- **HDMI/VGA** – Video output to monitors
- **Ethernet (RJ45)** – For network connection
- **Audio Jacks** – For speakers/microphones

9. Cables:

- **SATA Cables:** Connect storage devices to motherboard
- **Power Cables:** From SMPS to components
- **Data Cables:** For component communication

10. Computer Peripherals:

- **Input Devices:** Keyboard, mouse, scanner
- **Output Devices:** Monitor, printer, speakers
- **I/O Devices:** Touchscreens, external drives

11. Firmware:

Firmware is low-level software stored in non-volatile memory (ROM/Flash). It initializes hardware before handing over control to the operating system. Examples:

- **BIOS (Basic Input Output System)**
- **UEFI (Unified Extensible Firmware Interface)**

Result:

The internal and external hardware components of the computer system were successfully identified and studied. Their structure, connections, and individual functionalities were understood.

Experiment 2: Familiarization of Boot Process

Objective:

To understand and observe the various stages involved in the computer booting process, including POST, BIOS/UEFI execution, bootloader, and loading of the operating system.

Theory:

The **boot process** is the sequence of steps that a computer follows to start up and load the operating system. It begins when the system is powered on and ends when the OS is fully loaded and operational.

1. Power On:

When the power button is pressed, electricity is supplied to the motherboard and other hardware components by the **SMPS (Switched Mode Power Supply)**.

2. POST (Power-On Self Test):

POST is the **initial diagnostic test** performed by the system's firmware (BIOS/UEFI) to check whether essential hardware components like RAM, keyboard, CPU, and storage are connected and working properly.

If a failure occurs, error messages or beep codes are generated.

3. BIOS/UEFI Execution:

BIOS (Basic Input Output System) or **UEFI (Unified Extensible Firmware Interface)** is the firmware stored in the ROM/Flash memory on the motherboard. It performs the following tasks:

- Executes POST
- Detects and initializes hardware devices
- Displays initial screen and allows access to BIOS/UEFI settings
- Identifies the boot device (Hard Disk, SSD, USB, etc.)

4. Bootloader Execution:

Once the BIOS/UEFI identifies the boot device, it looks for the **bootloader** (like GRUB in Linux or Windows Boot Manager).

The bootloader is a small program responsible for:

- Loading the operating system kernel
- Providing options to choose between multiple operating systems (if any)

5. Loading the Operating System:

The operating system **kernel** is loaded into RAM. Control is handed over from the bootloader to the OS.

- In Windows, the process is managed by the **NT Loader (NTLDR)** or **bootmgr**
- In Linux, the **GRUB** bootloader loads the Linux kernel

Once the kernel is loaded:

- System drivers are initialized
- Background services (daemons in Linux, services in Windows) start
- Graphical user interface (GUI) is launched
- Login screen is presented to the user

Procedure:

1. Power on the system.

Observe the monitor for any initial messages or beep sounds indicating POST success or failure.

2. Watch for the BIOS/UEFI screen.

Press the required key (usually DEL, F2, ESC, or F10) to enter the BIOS/UEFI settings screen.

3. Inside BIOS/UEFI:

- o Navigate through menus using keyboard keys.
- o View information such as system date/time, hardware configuration, and boot order.

4. Exit BIOS and allow booting to continue.

The BIOS will hand over control to the bootloader on the selected boot device.

5. Observe the bootloader screen (if available):

- o In dual-boot systems, choose between OS options.
- o In single-boot systems, it auto-loads the OS.

6. Watch the OS loading screen.

Wait for the desktop or login screen to appear, indicating that the OS has successfully loaded.

7. Log in to the system and verify that all system services and GUI are functioning properly.

Result:

The different stages of the computer boot process were successfully observed and studied. A clear understanding was gained about how the system starts from power-on to operating system loading.

Experiment 3: Familiarizing Installation of Linux and Windows Operating Systems

Objective:

To understand the step-by-step process of installing both Linux and Windows operating systems on a computer system.

Theory:

Operating systems are essential software that manage hardware and software resources and provide services to computer programs. Installing an operating system is a fundamental task that prepares a computer for use.

There are two popular OS categories:

- **Windows OS** – A commercial operating system developed by Microsoft, known for its user-friendly interface.
- **Linux OS** – An open-source operating system available in various distributions like Ubuntu, Fedora, and Debian.

Installation involves booting the system from an installation medium, selecting partitions, formatting drives, and configuring settings.

System Requirements (General):

- **Processor:** 1 GHz or faster
 - **RAM:** 1 GB (32-bit) or 2 GB (64-bit) minimum
 - **Storage:** Minimum 20 GB of free disk space
 - **Installation Media:** USB/DVD containing the OS installer
-

Windows Installation Overview:

- Uses a GUI-based installer
 - Requires product key
 - Supports automatic driver installation
 - Default file system: NTFS
-

Linux Installation Overview (e.g., Ubuntu):

- Offers live session (try before install)
 - Allows dual-boot with Windows
 - File systems: ext4, swap, etc.
 - Supports manual and guided partitioning
 - Often doesn't require product key
-

Procedure:

A. Installation of Windows OS:

1. **Insert Windows bootable USB/DVD** and restart the system.
2. **Enter BIOS/UEFI settings** (DEL, F2, etc.) and set the boot device to USB/DVD.
3. **Save and exit BIOS** to begin Windows installation.
4. **"Press any key to boot from CD/DVD/USB"** when prompted.
5. On the setup screen, select:
 - o Language
 - o Time and currency format
 - o Keyboard methodClick **Next** and then **Install Now**.

6. Enter the **Windows product key** (or skip if not available).
 7. Accept the license terms and click **Next**.
 8. Choose **Custom: Install Windows only**.
 9. **Select the drive/partition** to install Windows:
 - o Format if needed
 - o Create new partitions if space is unallocated
 10. Click **Next**. The installation will begin.
 11. After installation, the system will restart several times.
 12. Configure initial setup:
 - Username, password
 - Privacy settings
 - Network preferences
-

B. Installation of Linux OS (Ubuntu):

1. **Insert Linux bootable USB/DVD** and restart the system.
2. Enter BIOS/UEFI and set USB/DVD as the first boot device.
3. Boot into the installer and choose:
 - o **"Try Ubuntu"** (optional to explore before installing)
 - o **"Install Ubuntu"** to start setup
4. Select language and keyboard layout.
5. Choose **installation type**:
 - o Erase disk and install Ubuntu
 - o Install alongside Windows (for dual boot)
 - o Manual partitioning (advanced)

6. Set up user details:
 - o Username and password
 - o Computer name
 7. Select the time zone.
 8. Confirm and begin installation.
 9. After installation, remove USB/DVD and reboot.
 10. Log in and verify successful installation.
-

Result:

The installation procedures of both Windows and Linux operating systems were successfully performed and observed. The step-by-step methods provided a clear understanding of OS setup, partitioning, and initial configuration.

OUTPUT

\$ ls -l

\$ ls -l

total 8

drwxr-xr-x 2 user user 4096 Apr 5 10:20 Documents

-rw-r--r-- 1 user user 27 Apr 5 10:15 file.txt

\$ mkdir newfolder

\$ mkdir newfolder

\$ ls

Documents file.txt newfolder

Experiment 4: Familiarizing Basic Unix/Linux Commands

Objective:

To explore and understand commonly used Unix/Linux commands for basic system operations such as file and directory handling, user management, and system monitoring.

Theory (with Syntax, Options & Explanation):

1. ls – List Directory Contents

- **Syntax:** `ls [options] [path]`
- **Common Options:**
 - `ls` – Lists files in current directory
 - `ls -l` – Long listing with details
 - `ls -a` – Includes hidden files (those starting with `.`)
 - `ls -lh` – Human-readable file sizes

Explanation:

Displays the contents of a directory.

2. mkdir – Make Directory

- **Syntax:** `mkdir [options] directory_name`
- **Common Options:**
 - `mkdir newdir` – Creates one directory
 - `mkdir -p dir1/dir2` – Creates nested directories

Explanation:

Used to create a new directory or nested structure.

\$ cp file.txt copy.txt

\$ cp file.txt copy.txt

\$ ls

copy.txt Documents file.txt newfolder

\$ mv file.txt renamed.txt

\$ mv file.txt renamed.txt

\$ ls

copy.txt Documents newfolder renamed.txt

\$ grep "user" /etc/passwd

\$ grep "user" /etc/passwd

user:x:1000:1000:User:/home/user:/bin/bash

3. cp – Copy Files and Directories

- **Syntax:** cp [options] source destination
- **Common Options:**
 - cp file1.txt file2.txt – Copies file1 to file2
 - cp -r dir1 dir2 – Recursively copies directories

Explanation:

Copies files and directories.

4. mv – Move or Rename Files

- **Syntax:** mv [options] source destination
- **Common Options:**
 - mv file1.txt folder/ – Moves file to folder
 - mv oldname.txt newname.txt – Renames file

Explanation:

Used to move or rename files and directories.

5. grep – Pattern Search in Text

- **Syntax:** grep [options] pattern [file]
- **Common Options:**
 - grep "main" file.c – Finds “main” in file.c
 - grep -i "word" file.txt – Case-insensitive search
 - grep -r "word" folder/ – Recursively search

Explanation:

Searches for patterns within files and prints matching lines.

\$ rmdir newfolder

\$ rmdir newfolder

\$ ls

copy.txt Documents renamed.txt

\$ chmod 755 renamed.txt

\$ chmod 755 renamed.txt

\$ ls -l renamed.txt

-rwxr-xr-x 1 user user 27 Apr 5 10:15 renamed.txt

\$ sudo useradd testuser

\$ sudo useradd testuser

\$ cat /etc/passwd | grep testuser

testuser:x:1001:1001::/home/testuser:/bin/bash

\$ sudo passwd testuser

\$ sudo passwd testuser

Enter new UNIX password:

Retype new UNIX password:

passwd: password updated successfully

6. rmdir – Remove Directory

- **Syntax:** rmdir [options] directory_name
- **Note:** Only removes **empty** directories.

Explanation:

Deletes a directory if it is empty.

7. chmod – Change File Permissions

- **Syntax:** chmod [options] mode file
- **Common Modes:**
 - chmod 755 file.sh – Gives execute permission
 - chmod u+x file – Adds execute permission to user

Explanation:

Modifies read, write, and execute permissions for users.

8. useradd – Add New User

- **Syntax:** sudo useradd [options] username
- **Options:**
 - -m – Creates home directory
 - -s /bin/bash – Sets login shell

Explanation:

Creates a new user account.

9. passwd – Change User Password

- **Syntax:** sudo passwd username

\$ history \$ history

```
1 ls
2 mkdir folder
3 cd folder
4 touch file.txt
5 cp file.txt file2.txt
```

\$ dmesg | grep usb

\$ dmesg | grep usb

```
[ 3.215678] usb 1-1: new high-speed USB device number 2 using ehci-pci
```

\$ cat /proc/cpuinfo

\$ cat /proc/cpuinfo

```
processor : 0
model name : Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz
cpu cores : 4
```

\$ uname -a

\$ uname -a

```
Linux ubuntu 5.15.0-105-generic #115~20.04.1-Ubuntu SMP x86_64 GNU/Linux
```

Explanation:

Sets or changes the password for a user account.

10. history – Command History

- **Syntax:** history [n]
- Shows last n commands used in the terminal.

Explanation:

Displays the list of previously executed commands.

11. dmesg – Display Kernel Messages

- **Syntax:** dmesg | less OR dmesg | grep usb

Explanation:

Displays system boot and hardware-related messages.

12. cpuinfo – Display CPU Information

- **Syntax:** cat /proc/cpuinfo

Explanation:

Shows detailed CPU details like model, speed, cores, etc.

13. uname – System Information

- **Syntax:** uname [options]
- **Options:**
 - -a – All information
 - -r – Kernel release
 - -n – Hostname

\$ du -sh Documents/

\$ du -sh Documents/

1.2M Documents/

\$ time ls

\$ time ls

Documents renamed.txt copy.txt

real 0m0.002s

user 0m0.001s

sys 0m0.001s

\$ write username

\$ write user

Hi, are you available for a quick chat?

<CTRL+D>

\$ sudo fdisk -l

\$ sudo fdisk -l

Disk /dev/sda: 500 GB

Device	Boot	Start	End	Sectors	Size	Id	Type
/dev/sda1	2048	1050623	1048576	512M	83	Linux	
/dev/sda2	1050624	976773119	975722496	465.5G	83	Linux	

Explanation:

Displays system and kernel-related information.

14. du – Disk Usage

- **Syntax:** du [options] [file/folder]
- **Options:**
 - du -h – Human-readable format
 - du -sh folder/ – Summary for folder

Explanation:

Estimates file space usage.

15. time – Time a Command

- **Syntax:** time command
- **Example:** time ls

Explanation:

Displays how long a command took to execute.

16. write – Send Message to User

- **Syntax:** write username [tty]
- Must be logged into the same system.

Explanation:

Sends a real-time message to another logged-in user.

17. fdisk – Partition Management

- **Syntax:** sudo fdisk [options] device
- **Example:** sudo fdisk -l

Explanation:

Lists or modifies disk partitions.

Result:

Basic Unix/Linux commands were successfully practiced in a terminal environment. The session provided hands-on experience in command-line navigation, file operations, system monitoring, and user management.

Experiment 5: Shell Programming in Linux (bash)

Objective:

To understand the basics of shell programming in Linux using the Bash shell and to write simple shell scripts for performing basic operations such as arithmetic, decision-making, and looping.

Theory:

Shell programming refers to writing a sequence of Linux commands in a file (script) that the shell can execute. It automates repetitive tasks and simplifies operations. The most commonly used shell in Linux is **bash** (Bourne Again SHell).

Basic Concepts:

1. Shebang (#!)

Used at the start of the script to indicate the script interpreter.

Example:

```
#!/bin/bash
```

2. Variables

Used to store data values. No space around =.

```
name="Raheena"
```

3. Input/Output

read is used to take user input, and echo is used to display output.

```
read name
```

```
echo "Hello $name"
```

4. Arithmetic Operations

You can perform arithmetic using `$(( ))` or `expr`.

```
sum=$((a + b))
```

```
> bash add_num.sh
```

```
4
```

```
3
```

```
7
```

5. Conditional Statements

Used for decision making.

```
if [ $a -gt $b ]
then
    echo "$a is greater"
else
    echo "$b is greater"
fi
```

6. Loops

for loop

```
o   for i in 1 2 3
o   do
o       echo $i
o   done
```

while loop

```
o   while [ $i -le 5 ]
o   do
o       echo $i
o       i=$((i + 1))
o   done
```

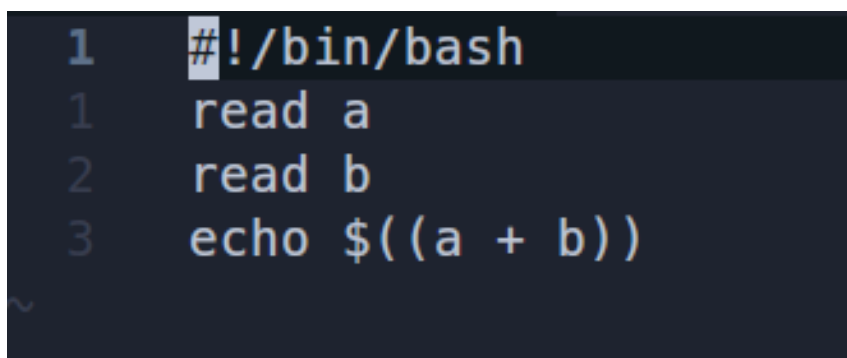
7. Comments

Begin with # and are ignored by the shell.

```
# This is a comment
```

Shell Programs

Add two numbers



```
1  #!/bin/bash
1  read a
2  read b
3  echo $((a + b))
```

```
college/32/lec 8 : main  
> bash recursion.sh  
5  
120
```

```
college/32/lec 8 : main  
> bash str_len.sh  
hello world  
11
```

write a Shell script that uses a recursive function to calculate factorial of given numbers

```
1  #!/bin/bash
1
2  factorial() {
3      local n="$1"
4      if [ "$n" -eq 0 ]; then
5          echo 1
6      else
7          local prev=$((n - 1))
8          local result=$(factorial "$prev")
9          echo $((n * result))
10     fi
11 }
12
13 read num
14 result=$(factorial "$num")
15 echo "$result"
```

length of given string

```
1  #!/bin/bash
1  read str
2  echo "${#str}"
```

1.creates an array of 5 elements

```
1  #!/bin/bash
1  my_array=("element1" "element2" "element3" "element4" "element5")
```

Result:

Sample shell scripts executed successfully.

Welcome to My Website

This is a simple static web page deployed locally using Live Server.

Experiment 6: Create a Web Page and Deploy on a Local Web Server

Objective:

To create a simple static web page using HTML and deploy it on a local web server using Visual Studio Code and the Live Server extension.

Theory:

♦ Web Page:

A **web page** is a document written in HTML that is viewed in a web browser. A static web page displays the same content for all users, whereas a dynamic web page can change content based on user interaction.

♦ HTML (HyperText Markup Language):

HTML is the standard markup language used to create web pages. It uses tags like `<html>`, `<head>`, `<title>`, `<body>`, `<h1>`, `<p>`, etc., to structure content.

♦ Local Web Server:

A **local web server** is a software that serves web content from your local machine, allowing developers to test websites before hosting them online.

Examples include:

- **Live Server** (VS Code extension)
- **Python HTTP Server**
- **XAMPP or WAMP (for PHP)**

♦ Live Server in VS Code:

This extension launches a development local server with live reload feature. It simplifies testing HTML/CSS/JS files without manual browser refresh.

Procedure:

Step 1: Set Up Project

- Open **Visual Studio Code**.
- Create a new folder named my website and open it in VS Code.
- Inside the folder, create a file named index.html.

Step 2: Write HTML Code

```
<!DOCTYPE html>

<html>

<head>

  <title>My First Web Page</title>

</head>

<body>

  <h1>Welcome to My Website</h1>

  <p>This is a simple static web page deployed locally using Live Server.</p>

</body>

</html>
```

Step 3: Install Live Server Extension

- Click on **Extensions icon** (Ctrl+Shift+X).
- Search for **Live Server** and click **Install**.

Step 4: Deploy the Web Page

- Right-click on index.html and click **"Open with Live Server"**.
 - The page will open in the browser at a local address like:
 - <http://127.0.0.1:5500/index.html>
-

Result:

A simple HTML web page was successfully created and deployed on a local web server using Live Server in Visual Studio Code. The web page opened in a browser and displayed the content as expected.

User Information Form

Name:

Email:

Age:

🌐 127.0.0.1:8080

Enter a valid email address

Experiment 7: Use JavaScript to Validate Forms

Objective:

To create a simple HTML form and validate user input using JavaScript before submission.

Theory:

♦ Form Validation:

Form validation ensures that the data entered by the user in a form is correct and complete before it is submitted to the server.

There are two types of validation:

- **Client-side validation** – Performed in the browser using JavaScript.
- **Server-side validation** – Performed on the server after the form data is submitted.

Using **JavaScript** for client-side validation improves user experience by providing instant feedback and reducing server load.

♦ Common Validations:

- Required fields should not be empty.
 - Email must be in proper format (e.g., name@example.com).
 - Password should meet length or complexity rules.
 - Age should be a number, within valid range.
-

Procedure:

Step 1: Create HTML Form

```
<!DOCTYPE html>
<html>
<head>
<title>Form Validation</title>
```

User Information Form

Name:

Email:

Age:

🌐 127.0.0.1:8080

Name must be filled out


```

<script>
    function validateForm() {
        let name = document.forms["myForm"]["username"].value;
        let email = document.forms["myForm"]["email"].value;
        let age = document.forms["myForm"]["age"].value;
        if (name == "") {
            alert("Name must be filled out");
            return false;
        }
        // Simple email regex check
        let emailPattern = /^[^ ]+@[^ ]+\.[a-z]{2,3}$/;
        if (!emailPattern.test(email)) {
            alert("Enter a valid email address");
            return false;
        }
        if (isNaN(age) || age < 1 || age > 100) {
            alert("Enter a valid age between 1 and 100");
            return false;
        }
        alert("Form submitted successfully!");
        return true;
    }
</script>
</head>
<body>
    <h2>User Information Form</h2>
    <form name="myForm" onsubmit="return validateForm()">
        Name: <input type="text" name="username"><br><br>
        Email: <input type="text" name="email"><br><br>
        Age: <input type="text" name="age"><br><br>
        <input type="submit" value="Submit">
    </form>
</body>
</html>

```

Result:

A form was created and validated using JavaScript. The form prevents submission when fields are empty or invalid, and shows appropriate error messages.

Image Slider Example



Experiment 8: Create an Image Slider Using HTML, CSS, and JavaScript

Objective:

To design a basic image slider (carousel) that allows users to navigate between images using “Previous” and “Next” buttons using HTML, CSS, and JavaScript.

Theory:

An **image slider** is a slideshow of images that can be navigated manually (using buttons) or automatically (timed transitions). It is widely used in websites to display image galleries, featured content, advertisements, etc.

In this experiment, we will use:

- **HTML** to structure the slider.
 - **CSS** to style and position the images and buttons.
 - **JavaScript** to implement the image switching functionality when buttons are clicked.
-

Procedure:

Step 1: Prepare 3 or more sample images in the same folder as your HTML file.

Step 2: Create the HTML structure

```
<!DOCTYPE html>
<html>
<head>
  <title>Image Slider</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin-top: 50px;
```



```

}

#slider {
  width: 500px;
  height: 300px;
  margin: auto;
  border: 2px solid #ccc;
  overflow: hidden;
  position: relative;
}

#slider img {
  width: 100%;
  height: 100%;
}

.btn {
  position: absolute;
  top: 45%;
  background-color: rgba(0,0,0,0.5);
  color: white;
  padding: 10px;
  border: none;
  cursor: pointer;
}

.prev {
  left: 0;
}

.next {
  right: 0;
}
</style>
</head>
<body>
<h2>Image Slider Example</h2>
<div id="slider">
  
  <button class="btn prev" onclick="prevImage()"><</button>
  <button class="btn next" onclick="nextImage()">>>/button>
</div>
<script>
  let images = ["image1.jpg", "image2.jpg", "image3.jpg"];
  let index = 0;

```



```
function showImage() {  
    document.getElementById('slide').src = images[index];  
}  
function nextImage() {  
    index = (index + 1) % images.length;  
    showImage();  
}  
function prevImage() {  
    index = (index - 1 + images.length) % images.length;  
    showImage();  
}  
</script>  
</body>  
</html>
```

Result:

An interactive image slider was created using HTML, CSS, and JavaScript. Users can view different images by clicking “Previous” and “Next” buttons.

Browse...

No file selected.

12

```
1 \documentclass{article}
2 \title{My First LaTeX Document}
3 \author{Peter Griffin}
4 \date{\today}
5 \begin{document}
6 \maketitle
7 \section{Introduction}
8 This is a simple LaTeX document.
9 \subsection{Formatting}
10 We can write \textbf{bold} text and \textit{italic} text.
11 \section{Lists}
12 \begin{itemize}
13 \item Apple
14 \item Banana
15 \item Mango
16 \end{itemize}
17 \section{Math Section}
18 Here is a formula: \(\displaystyle a^2 + b^2 = c^2\)
19 Another equation displayed:
20 \[
21 \frac{a+b}{c+d}
22 \]
23 \end{document}
24
```

Sample LaTeX

Friday, April 11, 2025

Abstract

The abstract goes here.

1 Text

Text, paragraphs and ligatures goes here.

2 Characters

Sample characters: \$ & % # _ { } ~ ^

3 Math

Sample math formulae: $f(x) = \int_{-\infty}^{\infty} \hat{f}(\xi) e^{2\pi i x \xi} d\xi$

4 Multicolumn

Column 1Column 2Column 3

5 Boxes

$\mbox{Sample box}$

Experiment 9: Familiarization of LaTeX – Basic Only

Objective:

To understand and use LaTeX for creating documents that include sections, lists, formatting, and mathematical expressions.

Theory:

LaTeX (pronounced *Lay-tech* or *Lah-tech*) is a high-quality typesetting system commonly used for technical and scientific documents. It allows users to focus on content while LaTeX handles formatting, especially for:

- Mathematical equations
- Scientific articles
- Research papers
- Reports and theses

LaTeX is a markup language like HTML but is specifically designed for documents.

Basic Structure of a LaTeX Document:

```
\documentclass{article}
\begin{document}
Your content goes here.
\end{document}
```

Common LaTeX Commands:

Task	Syntax Example
Title and Author	<code>\title{My Title}, \author{Author Name}, \maketitle</code>
Section	<code>\section{Section Name}</code>
Subsection	<code>\subsection{Subsection}</code>
Bold Text	<code>\textbf{Bold Text}</code>
Italic Text	<code>\textit{Italic Text}</code>
Bullet List	<code>\begin{itemize} \item Apple \item Mango \end{itemize}</code>
Numbered List	<code>\begin{enumerate} \item One \item Two \end{enumerate}</code>
Inserting Math	<code>\$a^2 + b^2 = c^2\$</code> (Inline), <code>\[a^2 + b^2 = c^2 \]</code> (Display)
Greek Letters	<code>\alpha</code> , <code>\beta</code> , <code>\theta</code>
Fractions	<code>\frac{a}{b}</code>
Square Root	<code>\sqrt{x}</code>

```
1 \documentclass{article}
2 \title{My First LaTeX Document}
3 \author{Peter Griffin}
4 \date{\today}
5 \begin{document}
6 \maketitle
7 \section{Introduction}
8 This is a simple LaTeX document.
9 \subsection{Formatting}
10 We can write \textbf{bold} text and \textit{italic} text.
11 \section{Lists}
12 \begin{itemize}
13   \item Apple
14   \item Banana
15   \item Mango
16 \end{itemize}
17 \section{Math Section}
18 Here is a formula: \(\displaystyle a^2 + b^2 = c^2\)
19 Another equation displayed:
20 \[
21 \frac{a+b}{c+d}
22 \]
23 \end{document}
24
```

Sample characters: \$ & % # _ { } ~ ^

3 Math

Sample math formulae: $f(x) = \int_{-\infty}^{\infty} \hat{f}(\xi) e^{2\pi i x \xi} d\xi$

4 Multicolumn

Column 1Column 2Column 3

5 Boxes

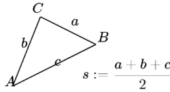
$\text{\mbox{Sample box}}$

6 Symbols

Sample symbols: $/\div\times-\pm\sqrt{\neg*}$

7 Picture

$$F = \sqrt{s(s-a)(s-b)(s-c)}$$



Sample picture:

Sample LaTeX Program:

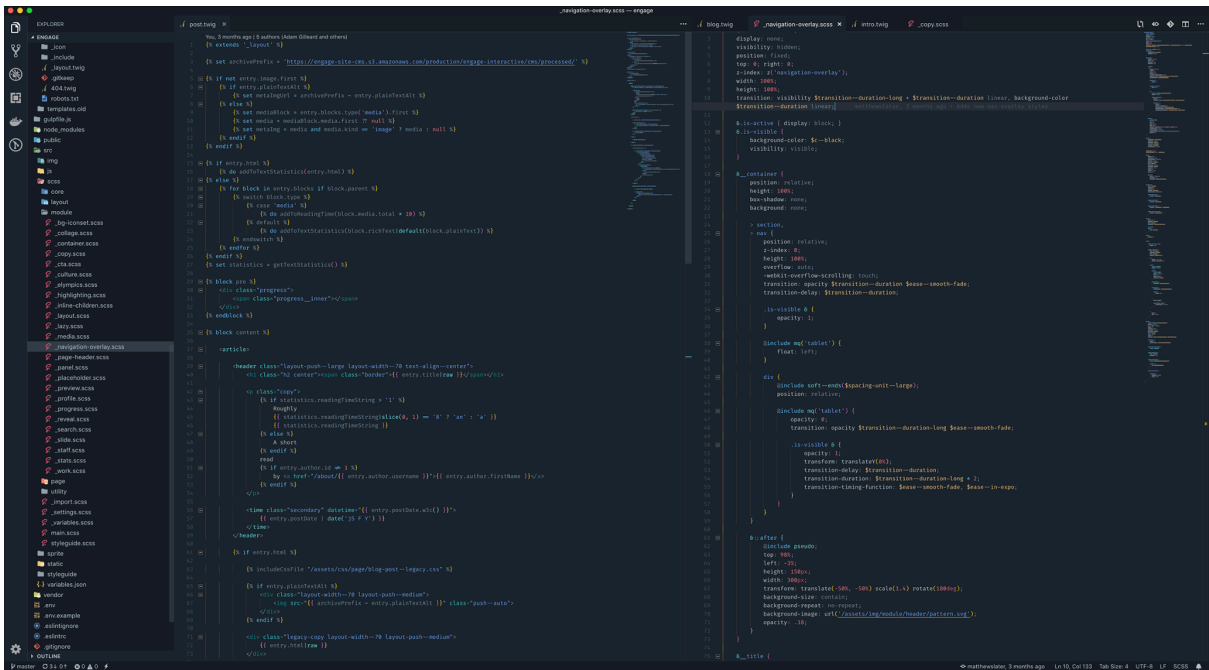
```
\documentclass{article}
\title{My First LaTeX Document}
\author{Peter Griffin}
\date{\today}
\begin{document}
\maketitle
\section{Introduction}
This is a simple LaTeX document.
\subsection{Formatting}
We can write \textbf{bold} text and \textit{italic} text.
\section{Lists}
\begin{itemize}
\item Apple
\item Banana
\item Mango
\end{itemize}
\section{Math Section}
Here is a formula:  $a^2 + b^2 = c^2$ 
Another equation displayed:

$$\frac{a+b}{c+d}$$

\end{document}
```

Result:

A basic LaTeX document was created and compiled successfully. The output included formatted text, lists, and mathematical equations.



Experiment 10: Familiarization of Development Environments – Visual Studio Code, Sublime Text, Atom

Objective:

To familiarize students with modern code editors like Visual Studio Code, Sublime Text, and Atom, and understand their basic features, extensions, and usage for programming and web development.

Theory:

A Development Environment is a space where developers write, test, and debug code efficiently. Popular environments offer:

- Syntax highlighting
- Auto-completion
- Debugging tools
- Git integration
- Extensions for various languages

Let's explore the three most widely used editors:

1. Visual Studio Code (VS Code)

Developer: Microsoft

Platform: Windows, macOS, Linux

Key Features:

- IntelliSense (smart code completion)
- Integrated terminal
- Extensions Marketplace
- Git support
- Live Server (for web development)
- Debugging tools

Getting Started:

- Launch VS Code
 - Open a folder (File > Open Folder)
 - Create a new file: File > New File → Save as index.html, script.js, etc.
 - Install Extensions: Ctrl+Shift+X → Search for *Live Server*, *Python*, *Prettier*, etc.
 - Open terminal: Ctrl+~
-

2. Sublime Text

Developer: Sublime HQ

Platform: Windows, macOS, Linux

Key Features:

- Lightweight and fast
- Multiple selections
- Command palette (Ctrl+Shift+P)
- Package Control (for plugins)
- Minimalistic interface

Getting Started:

- Open Sublime
 - Create and save a file
 - Install Package Control
 - Tools > Install Package Control
 - Use Ctrl+Shift+P to open command palette
 - Search for packages like *HTML-CSS-JS Prettify*, *Emmet*, etc.
-

3. Atom (Discontinued officially, but still used)

Developer: GitHub

Platform: Windows, macOS, Linux

Key Features:

- Open-source

- Built-in GitHub integration
- File system browser
- Packages and themes
- Markdown preview

Getting Started:

- Open Atom
- Create or open a project folder
- Use File > New File → Save as required
- Install packages from Settings > Install

Comparison Table:

Feature	VS Code	Sublime Text	Atom
Open Source	✓	✗ (Free trial)	✓
Git Integration	✓	With plugins	✓
Extensions	Rich Marketplace	Package Control	Package Browser
Performance	Fast	Very fast	Slower
Popular Cases	Use Web, C++	Python, Lightweight coding	Markdown, Git

Result:

Familiarized with three major development environments. Learned how to install, create files, add extensions/plugins, and start coding in each of them.

Experiment 11: Familiarizing Basic Networking Commands

Objective:

To understand and practice essential networking commands like `ifconfig`, `ping`, `traceroute`, `nslookup`, `ssh`, `scp`, `telnet`, and `ftp`, used to configure, test, and troubleshoot network-related issues in Linux systems.

Theory:

Networking commands help monitor, configure, and test computer network settings. They play a vital role in managing servers, connectivity, and file transfers.

1. `ifconfig`

Purpose: Displays and configures network interface parameters.

Syntax:

`ifconfig`

Common Options:

- `ifconfig eth0 up/down` – Enable/disable a network interface
- `ifconfig eth0 192.168.1.5` – Assign IP to interface

2. `ping`

Purpose: Checks connectivity to another host.

Syntax:

`ping [hostname/IP]`

Example:

`ping google.com`

Common Options:

- `-c [count]`: Number of echo requests to send
- `-i [interval]`: Time interval between pings

3. `traceroute`

Purpose: Traces the route packets take to reach a host.

Syntax:

`traceroute [hostname/IP]`

Example:

`traceroute google.com`

4. nslookup

Purpose: Queries DNS to obtain domain name or IP address mapping.

Syntax:

`nslookup [hostname/IP]`

Example:

`nslookup www.example.com`

5. ssh (Secure Shell)

Purpose: Securely connects to a remote system over a network.

Syntax:

`ssh username@hostname`

Example:

`ssh user@192.168.1.10`

6. scp (Secure Copy)

Purpose: Securely copies files between systems.

Syntax:

`scp source_file username@remote_host:/remote/directory`

Example:

`scp file.txt user@192.168.1.10:/home/user/`

7. telnet (Insecure, mostly deprecated)

Purpose: Connects to remote machines using the Telnet protocol.

Syntax:

`telnet [hostname/IP]`

Example:

`telnet towel.blinkenlights.nl`

8. ftp (File Transfer Protocol)

Purpose: Transfers files between systems using FTP protocol.

Syntax:

ftp [hostname]

Example:

ftp ftp.gnu.org

Result:

Successfully executed and understood the use of basic networking commands. These commands are essential for configuring interfaces, checking connectivity, resolving domain names, and transferring files securely.

